INTERNET-OF-THINGS AND EMBEDDED SYSTEM PRACTICE

Module Code: CSC83309

LECTURE

TUYISHIMIRE Joseph

tuyishimirejo1992@gmail.com

UNIVERSITY of KIGALI

INTERNET-OF-THINGS AND EMBEDDED SYSTEM PRACTICE

UNIVERSITY of KIGALI

Unit 2: Embedded Devices and Systems in IoT

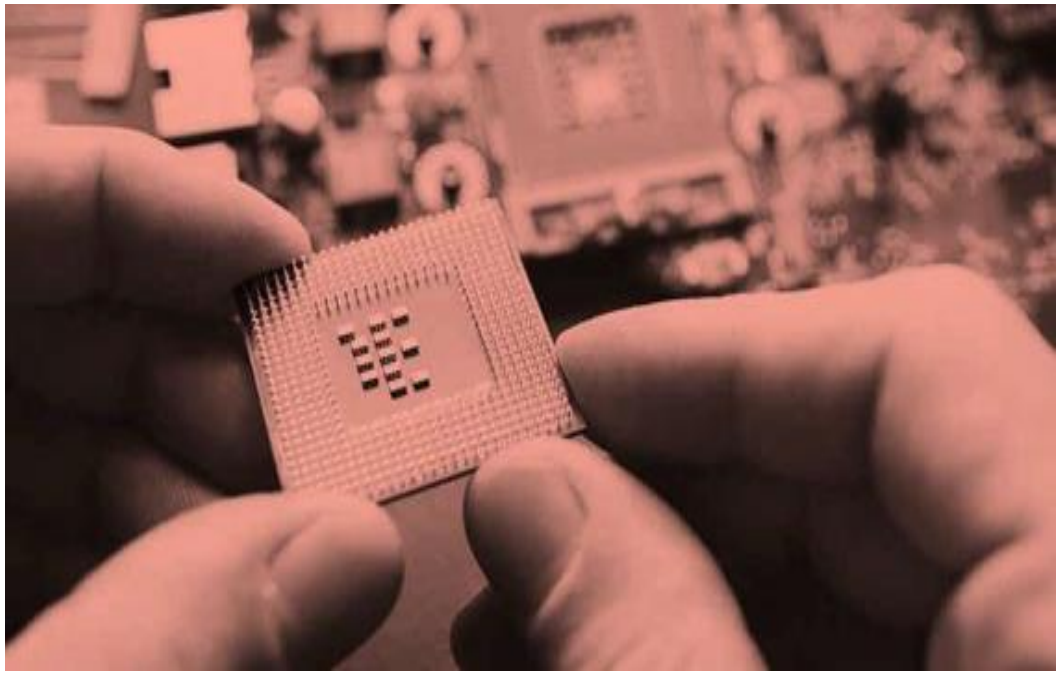
2.1 Introduction to Embedded Systems

2.2 Embedded System Hardware Components

2.3 Sensors and Actuators

2.4 Device Platforms for IoT

2.5 Arduino Programming Basics



UNIVERSITY of KIGALI

2.1 Introduction to Embedded Systems

Before going to the overview of Embedded Systems, Let's first know the two basic things i.e., embedded and system, and what actually do they mean.

- **System** is a set of interrelated parts/components which are designed/developed to perform common tasks or to do some specific work for which it has been created.
- Embedded means including something with anything for a reason. Or simply we can say something which is integrated or attached to another thing. Now after getting what actual systems and embedded mean we can easily understand what are Embedded Systems

Embedded System

Embedded system is a computational system that is developed based on an integration of both hardware and software in order to perform a given task. It can be said as a dedicated computer system has been developed for some particular reason. But it is not our traditional computer system or general-purpose computers, these are the Embedded systems that may work independently or attached to a larger system to work on a few specific functions. These embedded systems can work without human intervention or with little human intervention

UNIVERSITY of KIGALI

Examples of Embedded Systems

- Digital watches
- Washing Machine
- Toys
- Televisions
- Digital phones
- Laser Printer
- Cameras
- Industrial machines
- Electronic Calculators
- Automobiles
- Medical Equipment

UNIVERSITY of KIGALI

Types of Embedded Systems

Embedded system are classified based on the two factors

1. Performance and Functional Requirements
2. Performance of Micro-controllers

Based on Performance and Functional Requirements it is divided into 4 types as follows :

UNIVERSITY of KIGALI

Real-Time Embedded Systems

A Real-Time Embedded System is strictly time specific which means these embedded systems provides output in a particular/defined time interval. These type of embedded systems provide quick response in critical situations which gives most priority to time based task performance and generation of output. That's why real time embedded systems are used in defense sector, medical and health care sector, and some other industrial applications where output in the right time is given more importance. Further this Real-Time Embedded System is divided into two types

Real-Time Embedded Systems

- **Soft Real Time Embedded Systems -** In these types of embedded systems time/deadline is not so strictly followed. If deadline of the task is passed (means the system didn't give result in the defined time) still result or output is accepted.
- **Hard Real-Time Embedded Systems -** In these types of embedded systems time/deadline of task is strictly followed. Task must be completed in between time frame (defined time interval) otherwise result/output may not be accepted.

Examples :

- Traffic control system
- Military usage in defense sector
- Medical usage in health sector

UNIVERSITY of KIGALI

Stand Alone Embedded Systems

Stand Alone Embedded Systems are independent systems which can work by themselves they don't depend on a host system. It takes input in digital or analog form and provides the output.

Examples :

1. MP3 players
2. Microwave ovens
3. calculator

UNIVERSITY Of KIGALI

Networked Embedded Systems

Networked Embedded Systems are connected to a network which may be wired or wireless to provide output to the attached device. They communicate with embedded web server through network.

Examples :

1. Home security systems
2. ATM machine
3. Card swipe machine

UNIVERSITY of KIGALI

Mobile Embedded Systems

Mobile embedded systems are small and easy to use and requires less resources. They are the most preferred embedded systems. In portability point of view mobile embedded systems are also best.

Examples :

1. MP3 player
2. Mobile phones
3. Digital Camera

UNIVERSITY of KIGALI

Based on Performance and micro-controller it is divided into 3 types as follows :

UNIVERSITY Of KIGALI

Small Scale Embedded Systems

Small Scale Embedded Systems are designed using an 8-bit or 16-bit micro-controller.

- They can be powered by a battery.
- The processor uses very less/limited resources of memory and processing speed. Mainly these systems does not act as an independent system they act as any component of computer system but they did not compute and dedicated for a specific task.

UNIVERSITY Of KIGALI

Medium Scale Embedded Systems

Medium Scale Embedded Systems are designed using an 16-bit or 32-bit micro-controller. These medium Scale Embedded Systems are faster than that of small Scale Embedded Systems. Integration of hardware and software is complex in these systems. Java,c,c++ are the programming languages are used to develop medium scale embedded systems. Different type of software tools like compiler, debugger, simulator etc are used to develop these type of systems.

UNIVERSITY of KIGALI

Sophisticated or Complex Embedded Systems

Sophisticated or Complex Embedded Systems are designed using multiple 32-bit or 64-bit micro-controller. These systems are developed to perform large scale complex functions.

These systems have high hardware and software complexities. We use both hardware and software components to design final systems or hardware products.

Characteristics of an Embedded System

- **Performs specific task:** Embedded systems perform some specific function or tasks.
- **Low Cost:** The price of an embedded system is not so expensive.
- **Time Specific:** It performs the tasks within a certain time frame.
- **Low Power:** Embedded Systems don't require much power to operate.

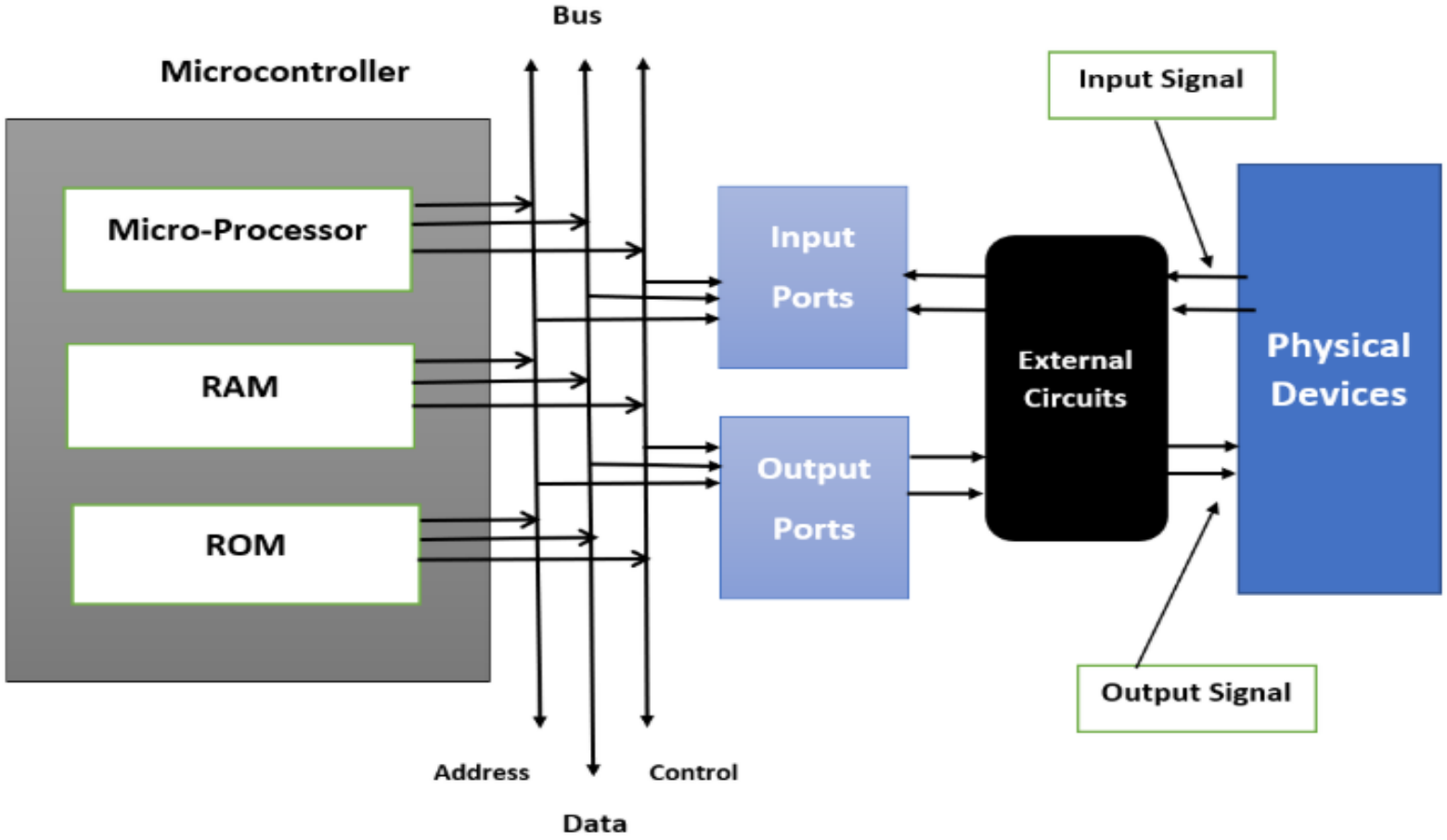
UNIVERSITY of KIGALI

- **High Efficiency:** The efficiency level of embedded systems is so high.
- **Minimal User interface:** These systems require less user interface and are easy to use.
- **Less Human intervention:** Embedded systems require no human intervention or very less human intervention.
- **Highly Stable:** Embedded systems do not change frequently mostly fixed maintaining stability.
- **High Reliability:** Embedded systems are reliable they perform tasks consistently well.

Manufacturable: The majority of embedded systems are compact and affordable to manufacture.

They are based on the size and low complexity of the hardware.

Block Structure of Embedded System



UNIVERSITY Of KIGALI

Advantages of Embedded System

- Small size.
- Enhanced real-time performance.
- Easily customizable for a specific application.

Disadvantages of Embedded System

- High development cost.
- Time-consuming design process.
- As it is application-specific less market available

How does an Embedded System Work?

Embedded systems operate from the combination of hardware and software that focuses on certain operations. An embedded system at its heart has microcontroller or microprocessor hardware on which user writes the code in form of software for control of the system. Here is how it generally works:

UNIVERSITY Of KIGALI

- **Hardware Layer:** Some of the hardware elements that are incorporated in an embedded system include the sensor, actuator, memory, current I/O interfaces as well as power supply. These components are interfaced with the micro controller or micro processor depending up on the input signals accepted.

- **Input/Output (I/O) Interfaces:** They to give the system input in form of data from sensors or inputs made by the users and the microcontroller processes the data received. The processed data is then utilized to coordinate the output devices such as displays, motors or communication modules.

UNIVERSITY of KIGALI

- **Firmware:** which is integrated within a system's hardware comprises of certain instructions to accomplish a task. Such software is often used for real time processing and is tuned to work in the most optimal manner on the system hardware.

- **Processing:** Depending on the given software and the input data received from the system's inputs the microcontroller calculates the appropriate output or response and manages the system's components.

Real-time Operation: Some of the most common systems are real time, this implies that they have the ability to process events or inputs at given time. This real time capability makes sure that the system accomplishes its intended function within stated time demands

2.2 Embedded System Hardware Components

UNIVERSITY of KIGALI

Embedded System Components

Due to the possibility of this question in interviews, there are 3 main components of embedded system. Thus, you should state that hardware, application-specific software, and real-time operating systems make up an embedded system. An embedded systems training prepares you to answer such queries. Hardware and software components both make up an embedded system. It also goes by the name "firmware" when it performs certain tasks. Every aspect of daily life has been impacted by the employment of embedded systems. The home appliances sector, the automotive sector, the agriculture sector, and the medical sector are a few examples of industries where it is employed. The major embedded system components include hardware and software.

Processor

Any embedded system's core component is the CPU. It determines how well the system works. There are three different processor types: 8-bit, 16-bit, and 32-bit CPUs. For embedded systems, smaller applications require fewer bits.

Large applications call for embedded systems with larger bit processors. The embedded system's brain is its processor. The following categories of processors are possible:

UNIVERSITY of KIGALI

A microcontroller

(MCU) is a compact integrated circuit designed to govern specific operations in embedded systems. It contains all the components of a computer on a single chip: the processor (CPU), memory (both RAM and ROM/Flash), input/output peripherals, timers, and communication interfaces. Microcontrollers are widely used in embedded systems due to their simplicity and cost-effectiveness in performing control-oriented tasks.

UNIVERSITY of KIGALI

Key Features of Microcontrollers

1. Integrated Peripherals: Built-in components like timers, ADCs (Analog to-Digital Converters), communication modules (I2C, SPI, UART).

2. On-Chip Memory: Includes both volatile memory (RAM) for runtime operations and non-volatile memory (Flash/EEPROM) for storing firmware.

3. Low Power Consumption: Optimized for energy-efficient applications, often with various low-power modes.

UNIVERSITY Of KIGALI

4. Cost-Effective: Suitable for mass production, especially in cost-sensitive embedded applications.

5. Real-Time Processing: Many microcontrollers are designed to handle real time tasks with real-time operating systems (RTOS) or bare-metal programming.

Applications of Microcontrollers in Embedded Systems:

- Home Appliances: Washing machines, microwave ovens, refrigerators
- Automotive Control Systems: Engine control units (ECU), ABS, airbags
- Consumer Electronics: Remote controls, calculators, smart home devices
- Industrial Control Systems: Motor controllers, industrial automation

Popular Microcontroller Families

- 8-bit Microcontrollers: AVR, PIC, Intel 8051
- 16-bit Microcontrollers: MSP430
- 32-bit Microcontrollers: ARM Cortex-M series, ESP32

A microprocessor

A microprocessor (MPU) is a central processing unit (CPU) on a single integrated circuit, typically lacking integrated memory, input/output peripherals, or other supporting components.

In embedded systems, microprocessors are used for more complex tasks and are often paired with external components, such as RAM, ROM, and input/output devices, to create a complete computing system

Key Features of Microprocessors:

- 1. High Processing Power: Microprocessors are designed for higher computational tasks, often found in applications that require multitasking, complex calculations, or high-speed processing.

- 2. External Components: Typically, microprocessors rely on external memory (RAM, ROM) and peripherals, giving designers flexibility in system configuration.

UNIVERSITY of KIGALI

- ✓ Complex Architecture: Capable of handling operating systems and largescale applications, often with multiple cores.
- ✓ Power Consumption: Higher than microcontrollers due to more complex instruction sets and higher clock speeds.
- ✓ Advanced Interfaces: Supports more advanced communication protocols and external memory interfaces for handling sophisticated tasks.

Applications of Microprocessors in Embedded Systems:

- Computers and Servers: Desktop computers, laptops, servers
- Smart Devices: Smartphones, tablets, and IoT devices
- Multimedia Systems: Audio/video processing in media players, gaming
- consoles
- Telecommunication Equipment: Routers, switches, modems
- Automotive Infotainment: Complex displays and infotainment systems in vehicles

UNIVERSITY Of KIGALI

Popular Microprocessor Families:

- ✓ ARM Processors: ARM Cortex-A series (used in mobile devices and embedded systems)
- ✓ Intel x86 Processors: Used in PCs, laptops, and high-performance embedded systems
- ✓ MIPS Processors: Common in networking devices like routers and switches
- ✓ RISC-V Processors: An open-source processor architecture used in embedded applications

UNIVERSITY of KIGALI

Memory

The microcontroller itself contains two different types of memory. RAM and ROM are these. ROM is a code memory, whereas RAM is a volatile memory.

ROM (Read-Only Memory)

ROM is non-volatile memory, meaning that its contents are retained even when the power is turned off.

In embedded systems, ROM is primarily used to store the firmware or program code that the system needs to run.

Types of ROM:

- Masked ROM (MROM): Programmed during the manufacturing process and cannot be modified. Used in mass production where the program code is fixed.
- Programmable ROM (PROM): Can be programmed once after manufacturing. Suitable for systems where the code needs to be written after production.

UNIVERSITY Of KIGALI

3.Erasable Programmable ROM (EPROM): Can be erased using ultraviolet (UV) light and reprogrammed. Used in systems that may need updates during development or deployment.

4. Electrically Erasable Programmable ROM (EEPROM): Can be electrically erased and reprogrammed, allowing for data to be rewritten without removing the chip. Common in embedded systems where small data updates are needed.

5. Flash Memory: A modern type of EEPROM that allows faster read/write cycles and supports multiple reprogramming cycles. It is widely used in embedded systems for storing both code and data.

Applications of ROM in Embedded Systems:

- Storing firmware or bootloader code.
- Storing fixed configuration settings.
- Storing data that rarely changes (e.g., calibration constants, lookup tables).

RAM (Random Access Memory)

RAM is volatile memory, meaning its contents are lost when the power is turned off. It is used to store data that the system needs to access quickly and frequently during runtime, such as variables, stack data, buffers, and temporary data that are used by the program.

Characteristics of ROM:

- Non-Volatile: Retains data even when the power is off.
- Read-Only: Typically used to store fixed program code, although certain
- types (EEPROM, Flash) allow for rewriting.
- Low Power Consumption: ROM does not require constant power to retain
- its contents.
- Slow Write Speeds: ROM types like EEPROM and Flash are slower to
- write compared to reading.

UNIVERSITY of KIGALI

Types of RAM:

1. Static RAM (SRAM): Retains data as long as power is supplied. It is faster and more expensive than Dynamic RAM (DRAM), but has lower capacity. Commonly used for caches or small working memory in embedded systems.
2. Dynamic RAM (DRAM): Requires periodic refreshing to retain data, but offers higher densities at a lower cost compared to SRAM. It is often used in systems where larger amounts of memory are needed.

Characteristics of RAM:

- Volatile: Loses its data when power is removed.
- Fast Access: Provides quick access to data, enabling high-speed execution of tasks.
- Temporary Storage: Holds data temporarily while the system is running (e.g., variables, stacks, and program counters).
- Low Capacity in Embedded Systems: Embedded systems typically use less RAM than general-purpose systems due to the specialized nature of their tasks

Applications of RAM in Embedded Systems:

- Data Storage During Execution: Storing variables, stack, and temporary data used by the processor during program execution.
- Buffering Data: Used as a buffer for data that is read from or written to peripherals, such as sensors or communication interfaces.
- Program Execution: Storing program instructions and data that are fetched and executed by the CPU.

UNIVERSITY Of KIGALI

Input/Output (I/O) Interfaces in Embedded Systems

I/O interfaces are critical for embedded systems, as they allow the system to interact with the external environment by sending and receiving data.

1. Digital Input/Output (GPIO)

General-Purpose Input/Output (GPIO) pins are simple digital pins that can be configured either as inputs or outputs. GPIOs are fundamental in embedded systems for interfacing with digital devices such as switches, LEDs, sensors, or relays.

- Digital Input: Reads the state of a digital signal (e.g., button press). The

input is typically binary, meaning it reads either a high (1) or low (0) signal.

- Digital Output: Sets the state of a digital pin to high (1) or low (0) to control external devices such as LEDs, motors, or relays.

UNIVERSITY of KIGALI

Examples of GPIO Use Cases:

- Digital Input: Reading the state of a button, switch, or motion sensor.
- Digital Output: Controlling an LED, relay, or buzzer.

Key Features:

- Configurable: Can be programmed as input or output.
- Voltage Levels: Typically operate at specific voltage levels (e.g., 3.3V or 5V).
- Debouncing: Required for stable input from mechanical devices like buttons.

Analog Input/Output (ADC/DAC)

Most real-world signals, such as temperature, sound, and light, are analog. Embedded systems need Analog-to-Digital Converters (ADC) to process these signals, and Digital-to-Analog Converters (DAC) for generating analog signals from digital values.

Analog Input (ADC)

An Analog-to-Digital Converter (ADC) converts continuous analog signals (like voltage) into discrete digital values. This is essential for interfacing sensors like temperature sensors, microphones, and pressure sensors.

- Resolution: Determined by the number of bits. A higher bit count (e.g., 10bit, 12-bit) allows for finer resolution.
- Sampling Rate: Determines how quickly the analog signal is sampled and converted into digital form.

UNIVERSITY of KIGALI

Analog Output (DAC)

A **Digital-to-Analog Converter (DAC)** converts digital signals (e.g., a sequence of binary numbers) into continuous analog signals. This is used to control devices that require analog signals such as speakers, motors, or variable voltage outputs.

Examples of ADC/DAC Use Cases:

- **ADC:** Reading sensor values like temperature, light intensity, or voltage.
- **DAC:** Outputting analog audio signals or controlling the speed of a motor with analog voltage.

UNIVERSITY Of KIGALI

Serial Communication Interfaces

Serial communication is a method of transmitting data one bit at a time over a communication channel or bus. Commonly used serial interfaces in embedded systems include:

UART (Universal Asynchronous Receiver/Transmitter)

- A widely used protocol for serial communication.
- **Asynchronous:** No clock signal is used; instead, data is transmitted with start and stop bits.
- **Full Duplex:** Can send and receive data simultaneously.
- **Use Cases:** Communication between microcontrollers, GPS modules, Bluetooth modules, and debugging (through RS232).

UNIVERSITY of KIGALI

SPI (Serial Peripheral Interface)

- A synchronous, high-speed communication protocol with a **master-slave architecture**.
- Uses separate lines for **MOSI** (Master Out Slave In), **MISO** (Master In Slave Out), **SCLK** (Serial Clock), and a **Chip Select** (CS) line for each slave.
- **Use Cases:** Communicating with high-speed peripherals like SD cards, displays, and sensors (e.g., accelerometers, gyroscopes).

UNIVERSITY Of KIGALI

I2C (Inter-Integrated Circuit)

- A two-wire (SDA for data, SCL for clock), synchronous communication protocol used for **multi-master and multi-slave** systems.
- Operates at slower speeds compared to SPI but supports multiple devices on the same bus.
- **Use Cases:** Connecting sensors, memory chips, and other peripherals in embedded systems.

UNIVERSITY of KIGALI

CAN (Controller Area Network)

- A robust communication protocol designed for **automotive** and industrial applications where reliability is crucial.
- **Multi-master:** Allows multiple devices to communicate on the same bus without a central controller.
- **Use Cases:** Automotive networks (engine control units, body control modules), industrial systems, and medical devices.

UNIVERSITY of KIGALI

Networking Interfaces

Networking interfaces allow embedded systems to communicate over local or wide-area networks, making it possible to connect devices to each other and to the internet.

- **Ethernet:**
  - A wired networking standard used for **high-speed** data transmission over local area networks (LAN).
  - Typically used in embedded systems requiring reliable, fast, and secure network communication, such as industrial automation, smart home systems, or networked medical devices.

UNIVERSITY of KIGALI

Wi-Fi:

- A wireless networking technology based on the IEEE 802.11 standard, commonly used for connecting embedded devices to the internet or local network.
- **Use Cases:** IoT devices, smart home appliances, wearable technology, and any system requiring remote monitoring/control.

UNIVERSITY Of KIGALI

Bluetooth:

- A short-range wireless communication protocol commonly used for **low-power** data transmission over short distances.
- **Use Cases:** Wireless peripherals (keyboards, mice), smart wearables (fitness trackers), and portable medical devices.

UNIVERSITY of KIGALI

ZigBee:

- A low-power, wireless communication standard designed for **low-data-rate** applications. It operates over a mesh network, making it highly scalable.
- **Use Cases:** Home automation, industrial automation, and smart energy systems (like smart meters).

LoRaWAN (Long Range Wide Area Network)

- A low-power, long-range wireless communication protocol for IoT and M2M (Machine-to-Machine) applications.
- **Use Cases:** Environmental monitoring, agricultural IoT applications, smart city infrastructure.

UNIVERSITY of KIGALI

Peripherals in Embedded Systems

Peripherals in embedded systems provide additional functionality to the core microcontroller or microprocessor, allowing it to interact with external devices or perform specialized tasks.

Timers (Real-Time Clock, RTC)

A **timer** is a peripheral that counts clock pulses to measure time intervals or trigger events after a specified duration. Timers are essential for scheduling tasks, generating delays, and producing periodic interrupts in embedded systems.

- **Real-Time Clock (RTC):**

  - An **RTC** is a dedicated timer used to keep track of the current time and date in real-world terms (hours, minutes, seconds, days, etc.).
  - Operates even when the system is powered off, typically using a small backup battery.
  - Used in systems that require accurate timekeeping, such as clocks, alarm systems, and data loggers.

UNIVERSITY of KIGALI

Common Applications of Timers:

- **Task Scheduling:** Running periodic tasks in real-time operating systems (RTOS).
- **Time Stamping:** Keeping accurate records of events (e.g., data logging).
- **Delays and Timeouts:** Implementing delays or timeouts in communication protocols.

UNIVERSITY of KIGALI

Counters

Counters are peripherals that count the number of events or signals in an embedded system. They are typically used to count external events such as pulses from sensors, or internal events like clock cycles.

- **Key Features of Counters:**

  - **Up/Down Counting:** Counters can count up or down based on the application requirements.
  - **Event-Based:** Can count events such as button presses, pulses from rotary encoders, or external sensor inputs.

UNIVERSITY of KIGALI

Common Applications of Counters:

- **Frequency Measurement:** Counting the number of pulses in a specific time to calculate frequency.
- **Event Counting:** Keeping track of external events like revolutions in a motor or distance traveled by a vehicle.
- **Timers:** Often used in conjunction with timers for measuring time intervals between events.

PWM Controllers (Pulse Width Modulation)

A **Pulse Width Modulation (PWM) controller** is used to generate digital signals with varying duty cycles. PWM is often used to control devices that require variable power, such as motors, LEDs, and heating elements.

- **Key Features of PWM:**

  - **Duty Cycle:** The percentage of time the signal is high within a single period. Adjusting the duty cycle controls the amount of power delivered to the device.

  - **Frequency:** The number of times the signal oscillates between high and low per second.

UNIVERSITY Of KIGALI

Common Applications of PWM:

Motor Control: Adjusting the speed of DC motors by varying the duty cycle.

- **LED Dimming:** Controlling the brightness of LEDs.
- **Audio Signal Generation:** Generating analog signals from digital signals by filtering the PWM output.
- **Power Control:** Adjusting the power delivered to heaters or voltage regulators.

UNIVERSITY of KIGALI

DMA Controllers (Direct Memory Access)

A **Direct Memory Access (DMA) controller** allows peripherals to directly transfer data to or from memory without the involvement of the CPU. This frees up the CPU for other tasks and improves system performance, especially in applications that require large data transfers.

- **Key Features of DMA:**

  - **Data Transfer:** Transfers data between memory and peripherals or between two memory locations without CPU intervention.
  - **Channels:** DMA controllers typically have multiple channels that can be used for different peripherals simultaneously.
  - **Burst Mode:** Transfers data in bursts to optimize performance and reduce bus contention.

Common Applications of DMA:

- **Data Streaming:** Transferring data between sensors (e.g., ADC) and memory for continuous data collection.
- **Peripheral Communication:** Handling high-speed data transfer between communication interfaces (UART, SPI, I2C) and memory.
- **Audio/Video Systems:** Streaming audio or video data without burdening the CPU.

Watchdog Timers

A **watchdog timer** is a safety feature used to reset the system if it becomes unresponsive or enters an unexpected state. The watchdog timer must be periodically reset by the system; if not, it assumes the system has crashed and performs a system reset.

- **Key Features of Watchdog Timers:**
- **Timeout Period:** If the system fails to reset the watchdog within a specific timeout period, a reset is triggered.
- **System Stability:** Ensures the system can recover from crashes, infinite loops, or other software malfunctions

UNIVERSITY Of KIGALI

Common Applications of Watchdog Timers:

- **System Recovery:** Automatically restarting embedded systems when they become unresponsive.
- **Fail-Safe Operations:** Used in critical systems (e.g., automotive, industrial control) to ensure that the system resets and returns to a known state after a failure.

UNIVERSITY Of KIGALI

Drivers

Drivers are low-level software components that manage communication between the operating system or firmware and hardware peripherals. Each peripheral requires its driver to handle communication, initialization, and data transfer between the peripheral and the processor.

- **Key Features of Drivers:**

  - **Hardware Abstraction:** Drivers abstract the hardware details, making it easier for software to interface with peripherals.
  - **Initialization and Control:** Drivers handle the setup and control of peripherals (e.g., configuring timers, ADCs, or communication interfaces).
  - **Data Handling:** Manage data transfer between peripherals and the system memory, including buffering and error checking.

Power Supply in Embedded Systems

The **power supply** is a critical component of any embedded system, ensuring that the system and its components receive the appropriate voltage and current for reliable operation.

UNIVERSITY Of KIGALI

Key Components of Power Supply in Embedded Systems

Power Source: The type of power source used in an embedded system depends on the application. Common power sources include:

- **Batteries:** Used in portable or low-power devices (e.g., wearables, IoT devices).
- **Mains Power (AC):** Common in systems that are always plugged into an electrical outlet (e.g., industrial control systems, home appliances).
- **Renewable Energy:** Solar panels or other renewable sources for remote or environmentally conscious systems (e.g., outdoor sensors, environmental monitoring).

UNIVERSITY Of KIGALI

Voltage Regulators:

Voltage Regulators: Embedded systems often need specific voltages for their components. Voltage regulators ensure that a stable, regulated voltage is supplied to the system despite variations in input voltage or load conditions. Types of voltage regulators include:

- **Linear Regulators:** Simple, cost-effective regulators that provide a constant output voltage by dissipating excess energy as heat. They are not efficient but are easy to implement (e.g., 7805 regulator for 5V output).
- **Switching Regulators (DC-DC Converters):** More efficient than linear regulators, these converters use switching techniques to step up or step down the input voltage. They are widely used in systems where efficiency and heat dissipation are critical.

- **Power Management Integrated Circuits (PMICs):** PMICs are specialized ICs that manage various power-related functions such as voltage regulation, battery charging, power sequencing, and power monitoring. These are used in systems with multiple power rails or complex power requirements (e.g., mobile phones, IoT devices).
- **Energy Harvesting Modules:** In some embedded systems, energy harvesting modules are used to generate power from ambient sources such as light (solar), vibration, or thermal energy. These are often used in ultra-low-power systems where battery replacement is not feasible (e.g., wireless sensor networks).
- **Capacitors and Filters:** Capacitors are used to smooth out fluctuations in power supply (ripple) and filter noise. They are essential for providing clean and stable power to sensitive components such as microcontrollers, sensors, and communication modules.

Types of Power Supplies for Embedded Systems

- **Battery Power:**
  - **Primary Batteries:** Non-rechargeable batteries such as alkaline or lithium coin cells, used in low-power devices where battery replacement is acceptable (e.g., remote controls, some IoT sensors).
  - **Secondary Batteries (Rechargeable):** Rechargeable batteries such as lithium-ion, lithium-polymer, or nickel-metal hydride (NiMH), used in devices where recharging is necessary (e.g., smartphones, wearables).
  - **Battery Management Systems (BMS):** Circuits that manage battery charging, discharging, and health monitoring to ensure safe operation.

AC to DC Power Supplies:

- Used in devices that plug into wall outlets, such as home appliances or industrial controllers.
- **AC-DC Converters:** Convert alternating current (AC) from mains to direct current (DC) for use by the embedded system.
- **Switched-Mode Power Supplies (SMPS):** A type of AC-DC converter known for high efficiency, widely used in devices with varying power demands.

- **USB Power:**
  - **USB Power Delivery:** Embedded systems such as microcontroller boards (e.g., Arduino, Raspberry Pi) can draw power from a USB port. USB power is commonly used in development and prototyping systems or for low-power devices.
- **Renewable Power Sources:**
  - **Solar Power:** Often used in outdoor or remote systems (e.g., environmental monitoring stations) where constant access to mains power is not available.
  - **Energy Harvesting:** Systems that derive power from ambient sources (e.g., piezoelectric or thermoelectric generators) typically operate at very low power levels.

Real-Time Operating System (RTOS):

A **Real-Time Operating System (RTOS)** is a specialized operating system designed to manage hardware resources and ensure that critical tasks are executed within strict time constraints. The key features of an RTOS include:

- **Determinism:** The ability to execute tasks predictably within a known time frame.
- **Priority-based Scheduling:** RTOS assigns priorities to tasks, ensuring that high-priority tasks are executed first.
- **Concurrency:** RTOS manages multiple tasks concurrently, often running them in parallel on different processors.

UNIVERSITY Of KIGALI

- **Minimal Latency:** It guarantees that the time taken to respond to external events (interrupts) is minimal.
- **Reliability and Stability:** RTOS must operate reliably over long periods, particularly in embedded systems like automotive, medical devices, or industrial automation.

Interrupt Handling:

Interrupts are signals that alert the CPU to immediate tasks requiring attention, temporarily halting ongoing processes. **Interrupt handling** in an RTOS is critical for maintaining real-time performance. Key aspects include:

- **Interrupt Service Routine (ISR):** A function or routine executed in response to an interrupt. It must be as short as possible to minimize interrupt latency.
- **Preemptive Handling:** RTOS can preempt the current task to handle higher-priority interrupts.
- **Interrupt Prioritization:** RTOS allows prioritization of interrupts so that more critical events are handled first.

- **Context Switching:** When an interrupt occurs, RTOS must save the current state of the running task and restore it after the ISR completes.
- **Latency:** The time between the arrival of the interrupt and the execution of its handler. Low latency is crucial in real-time systems.

Error Handling:

Error handling in RTOS involves identifying, managing, and recovering from faults or exceptional conditions without disrupting system operation. The key aspects of RTOS error handling are:

- **Error Detection:** RTOS must detect system errors such as task overruns, hardware failures, or memory issues.
- **Graceful Degradation:** Instead of complete system failure, the RTOS should provide mechanisms for the system to continue functioning, perhaps with limited capability.
- **Recovery Mechanisms:** RTOS supports mechanisms like retries, resets, or safe shutdowns to recover from errors.

- **Logging and Diagnostics:** Error logging is crucial for diagnosing system behavior and improving future reliability.

- **Watchdog Timers:** A common error-handling feature where the system resets itself if a task takes too long or behaves unexpectedly.

Scheduling:

Scheduling in RTOS is responsible for determining the order in which tasks are executed. RTOS uses various scheduling algorithms based on task priorities, deadlines, and resource availability. Key types of scheduling include:

- **Priority-based Scheduling:** Each task is assigned a priority, and the scheduler ensures that the highest-priority task is executed first.
- **Preemptive Scheduling:** The RTOS can interrupt lower-priority tasks if a higher-priority task becomes ready to run.
- **Round-Robin Scheduling:** Each task gets a fixed time slice to run, ensuring that all tasks receive CPU time.

UNIVERSITY Of KIGALI

- **Rate Monotonic Scheduling (RMS):** A fixed-priority algorithm where shorter tasks get higher priority, often used in real-time systems.
- **Earliest Deadline First (EDF):** A dynamic priority algorithm where tasks with the nearest deadline are given higher priority.
- **Task Synchronization:** Scheduling must handle resource sharing and synchronization between tasks to avoid issues like deadlocks or race conditions

CPU Utilization

- The final term to be defined is a critical measure of real-time system performance
- Because the CPU continues to execute instructions as long as power is applied, it will more or less frequently execute instructions that are not related

to the fulfillment of a specific deadline

- The measure of the relative time spent doing non-idle processing indicates

how much real-time processing is occurring

UNIVERSITY Of KIGALI

CPU Utilization Factor

The CPU utilization or time-loading factor, U, is a relative measure of the non-idle processing taking place

CPU Utilization Zones

- A system is said to be time-overloaded if $U > 100\%$
- Systems that are too highly utilized are problematic —additions, changes, or corrections cannot be made to the system without risk of time-overloading
- On the other hand, systems that are not sufficiently utilized are not necessarily cost-effective —The system was over-engineered and that costs could likely be reduced with less expensive hardware
- While a utilization of 50% is common for new products, 80% might be acceptable for systems that do not expect growth
- However, 70% as a target for U is one of the potentially useful results in the theory of real-time systems where tasks are periodic and independent

UNIVERSITY Of KIGALI

TAB 3. CPU Utilization Zones

CPU utilization [in %]	Zone Type	Typical Application
< 26	Unnecessarily safe	Various
26 – 50	Very safe	Various
51 – 68	Safe	Various
69	Theoretical limit	Embedded systems
70 – 82	Questionable	Embedded systems
83 – 99	Dangerous	Embedded systems
100	Critical	Marginally stressed system
> 100	Overloaded	Stressed system

Calculation of the CPU utilization

- U is calculated by summing the contribution of utilization factors for each task
- Suppose a system has$n \geq 1$ periodic tasks, each with an execution period of p_i
- If task i is known to have a worst-case execution time of e_i, then the utilization factor,u_i, for task i is given in Equation (1)

$$u_i = \frac{e_i}{p_i} \tag{1}$$

- The overall system utilization factor, U, is given by Equation (2)

$$\begin{aligned} U &= \sum_{n}^{i=1} u_i \\ &= \sum_{n}^{i=1} \frac{e_i}{p_i} \end{aligned} \tag{2}$$

- In practice, the determination of e_i can be difficult, in which case estimation or measuring must be used
- For aperiodic and sporadic tasks, u_i is calculated by assuming a worst-case execution period

UNIVERSITY Of KIGALI

Example—calculation of CPU utilization

An individual elevator controller in a bank of elevators has the following tasks with execution periods of p_i and worst-case execution times of $e_i \in [1, 2, 3, 4]$ as shown in Table 4:

- Task 1—Communicate with the group dispatcher.
- Task 2—Update the car position information and manage floor-to-floor runs as well as door control.
- Task 3—Register and cancel car calls.
- Task 4—Miscellaneous system supervisions.

The CPU utilization is computed as Equation (3)

$$U = \sum_{n}^{i=1} \frac{e_i}{p_i} \tag{3}$$

We note that $U = 31\%$ —which is a very safe zone

TAB 4. CPU utilization

i	e_i	p_i	$\frac{e_i}{p_i}$
1	17	500	0.03
2	4	25	0.16
3	1	75	0.01
4	20	200	0.10
U			**0.31%**

END